CRM Platform - Security

Version: 1.0
Status: Draft
Last Updated: June 2026

Table of Contents

1. Purpose
2. Security Objectives
3. Security Principles
4. Authentication
5. Authorization
6. Password Security
7. Token Management
8. API Security
9. Data Protection
10. Secrets Management
11. File Security
12. Infrastructure Security
13. Database Security
14. Logging & Auditing
15. Security Headers
16. Dependency Security
17. Security Monitoring
18. Incident Response
19. Security Checklist
20. Future Enhancements

---

1. Purpose

This document defines the security architecture, policies, and best practices for the CRM Platform.

Security is integrated into every layer of the platform, including:

- Frontend
- API Gateway
- Microservices
- Database
- Infrastructure
- CI/CD Pipeline

The objective is to ensure confidentiality, integrity, availability, and traceability while following modern security best practices.

---

2. Security Objectives

The platform is designed to achieve the following security goals:

- Protect user identities
- Secure APIs
- Prevent unauthorized access
- Protect sensitive business data
- Ensure secure communication between services
- Maintain comprehensive audit trails
- Detect and respond to security incidents
- Support future compliance requirements

---

3. Security Principles

The CRM Platform follows these core security principles.

Security by Design

Security is considered during system design rather than added later.

Least Privilege

Users, services, and infrastructure receive only the permissions necessary to perform their tasks.

Defense in Depth

Multiple security layers protect the platform.

Example:

```
Internet
    |
Firewall
    |
API Gateway
    |
Authentication
```

```
    |
Authorization
    |
Business Logic
    |
Database
```

Zero Trust

Every request must be authenticated and authorized regardless of its origin.

Secure Defaults

The default configuration should always be the most secure configuration.

---

4. Authentication

Authentication is managed exclusively by the **Identity Service**.

Supported authentication methods:

- Username and Password
- JWT Access Tokens
- Refresh Tokens

Authentication Flow:

```
User
 |
 ▼
Identity Service
 |
 ▼
Validate Credentials
 |
 ▼
Generate JWT
 |
 ▼
Return Access Token
 |
```

```
▼
Frontend
```

The API Gateway validates JWT tokens before forwarding requests to backend services.

---

5. Authorization

The platform implements **Role-Based Access Control (RBAC).**

Example roles:

Role	Description
Admin	Full platform access
Manager	Team management
Sales	CRM operations
Viewer	Read-only access

Permissions are evaluated inside each microservice after authentication.

Example:

```
Admin
 ├── Create Users
 ├── Delete Users
 ├── Manage Roles
 └── Full CRM Access

Sales
 ├── Create Leads
 ├── Update Leads
 └── View Opportunities
```

---

6. Password Security

Passwords are never stored in plain text.

Hashing Algorithm

Preferred:

- Argon2

Supported:

- bcrypt

Password Policy

Minimum requirements:

- 12 characters
- Uppercase letter
- Lowercase letter
- Number
- Special character

Passwords should never be logged or transmitted in plain text.

---

7. Token Management

Access Token

- Short-lived (15–30 minutes)
- Included in every request
- Stored securely by the client

Refresh Token

- Long-lived
- Stored securely
- Used only to obtain new access tokens
- Revocable

Authorization header:

```
Authorization: Bearer <access_token>
```

---

8. API Security

Every API request must include:

- HTTPS
- JWT Authentication
- Input Validation
- Authorization Check

Additional protections:

- Request size limits
- Rate limiting
- Correlation IDs
- Standardized error responses
- Request logging

Example headers:

```
Authorization: Bearer <JWT>
Content-Type: application/json
Accept: application/json
X-Correlation-ID: <UUID>
```

---

9. Data Protection

Sensitive information includes:

- Password hashes
- Refresh tokens
- JWT signing keys
- API keys
- Personally Identifiable Information (PII)

Protection mechanisms:

- TLS encryption in transit
- Encryption at rest where supported
- Principle of least privilege
- Secure backups

---

10. Secrets Management

Secrets must never be committed to source control.

Examples:

- Database credentials
- RabbitMQ credentials
- JWT signing keys
- AWS credentials
- SMTP credentials

Development:

```
.env
```

Production:

- Kubernetes Secrets
- AWS Secrets Manager (Future)

Secrets should be rotated periodically.

---

11. File Security

The File Service is responsible for secure file handling.

Requirements:

- Validate MIME type
- Restrict file size
- Generate unique filenames
- Prevent directory traversal
- Store files in Amazon S3
- Restrict public access
- Scan uploads for malware (Future)

---

12. Infrastructure Security

Infrastructure controls include:

- Docker containers run as non-root users
- Minimal container images
- Firewall configuration
- Secure Kubernetes configuration
- Network segmentation
- Restricted administrative access

SSH access should be limited to authorized administrators.

---

13. Database Security

Database access follows the **Schema-per-Service** model.

Rules:

- Services only access their own schema
- No shared database users
- Least-privilege permissions
- Parameterized queries only
- Regular backups

Connections should always use TLS where supported.

---

14. Logging & Auditing

Security-relevant events must be logged.

Examples:

- User login
- Login failure
- Password reset
- Role changes
- User creation
- File uploads
- Permission changes

Sensitive data must never appear in logs.

Do **not** log:

- Passwords
- Access tokens
- Refresh tokens
- API keys
- Secrets

Audit logs should be immutable.

---

15. Security Headers

Recommended HTTP response headers:

```
Strict-Transport-Security

Content-Security-Policy

X-Content-Type-Options

X-Frame-Options

Referrer-Policy

Permissions-Policy
```

These headers reduce the risk of common web attacks.

---

16. Dependency Security

All dependencies must be:

- Regularly updated
- Version locked
- Vulnerability scanned
- Reviewed before adoption

Security tools may include:

- npm audit
- pip-audit
- Dependabot

- GitHub Security Advisories

---

17. Security Monitoring

Security monitoring is achieved through:

- Prometheus
- Grafana
- Loki
- Jaeger

Alerts should be generated for:

- Excessive login failures
- Suspicious API activity
- Unauthorized access attempts
- Service failures
- Infrastructure anomalies

---

18. Incident Response

The incident response process consists of:

1. Detect
2. Contain
3. Investigate
4. Eradicate
5. Recover
6. Conduct a post-incident review

Audit logs and monitoring data should support forensic investigations.

---

19. Security Checklist

Before deploying a service, verify:

- JWT authentication implemented
- RBAC enforced
- Input validation completed
- Secrets externalized
- HTTPS enabled

- Logging configured
- Rate limiting enabled
- Dependency scan passed
- Security headers configured
- Database permissions reviewed

---

20. Future Enhancements

The following security features are planned for future releases:

- Multi-Factor Authentication (MFA)
- OAuth 2.0
- OpenID Connect (OIDC)
- Single Sign-On (SSO)
- Passkeys (WebAuthn)
- AWS Secrets Manager integration
- Security Information and Event Management (SIEM)
- Automated malware scanning
- Key rotation automation
- Open Policy Agent (OPA) for centralized authorization

Related Documents

- project-overview.md
- system-architecture.md
- api-standards.md
- database-design.md
- deployment.md